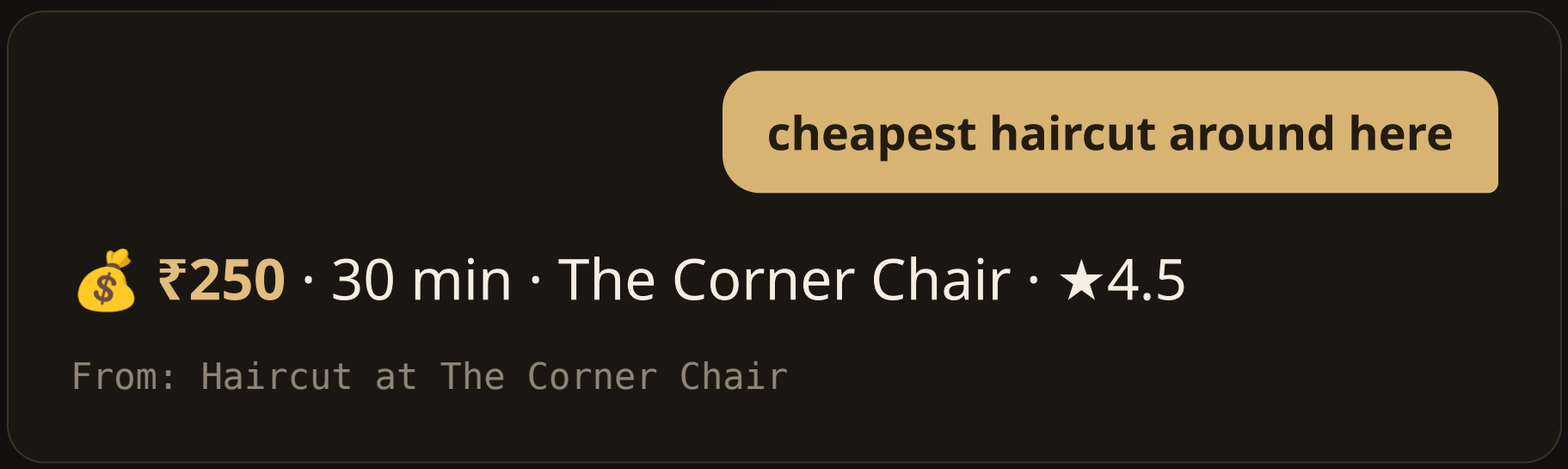
BUILD NOTE · TRIMLY

A booking chatbot that can’t make up a price.

Or a free slot. Or today’s revenue.
Nine slides on grounding a chat layer in your own database.



THE TRAP

Chat + database = confident guesses.

FAILURE ONE

“A haircut is ₹200.”

The price sounds plausible, comes from nowhere, and the customer finds out at the counter.

FAILURE TWO

“How many bookings today?” → a *this week* headline.

The numbers were today’s. The label wasn’t. Looks fine in a screenshot, wrong in a review.

Both are worse than “I don’t know”.

THE RULE

Retrieve first, write second.

01 **Question**

02 **Intent**
weighted keywords — “how many bookings” beats “booking”

03 **Retrieve rows**
BM25 over documents built from the database, this request

04 **Compose the sentence**
prices from price_paise, slots from the availability engine

05 **Cite the rows**
every answer carries its sources

The sentence is never written before the data arrives.

RETRIEVAL

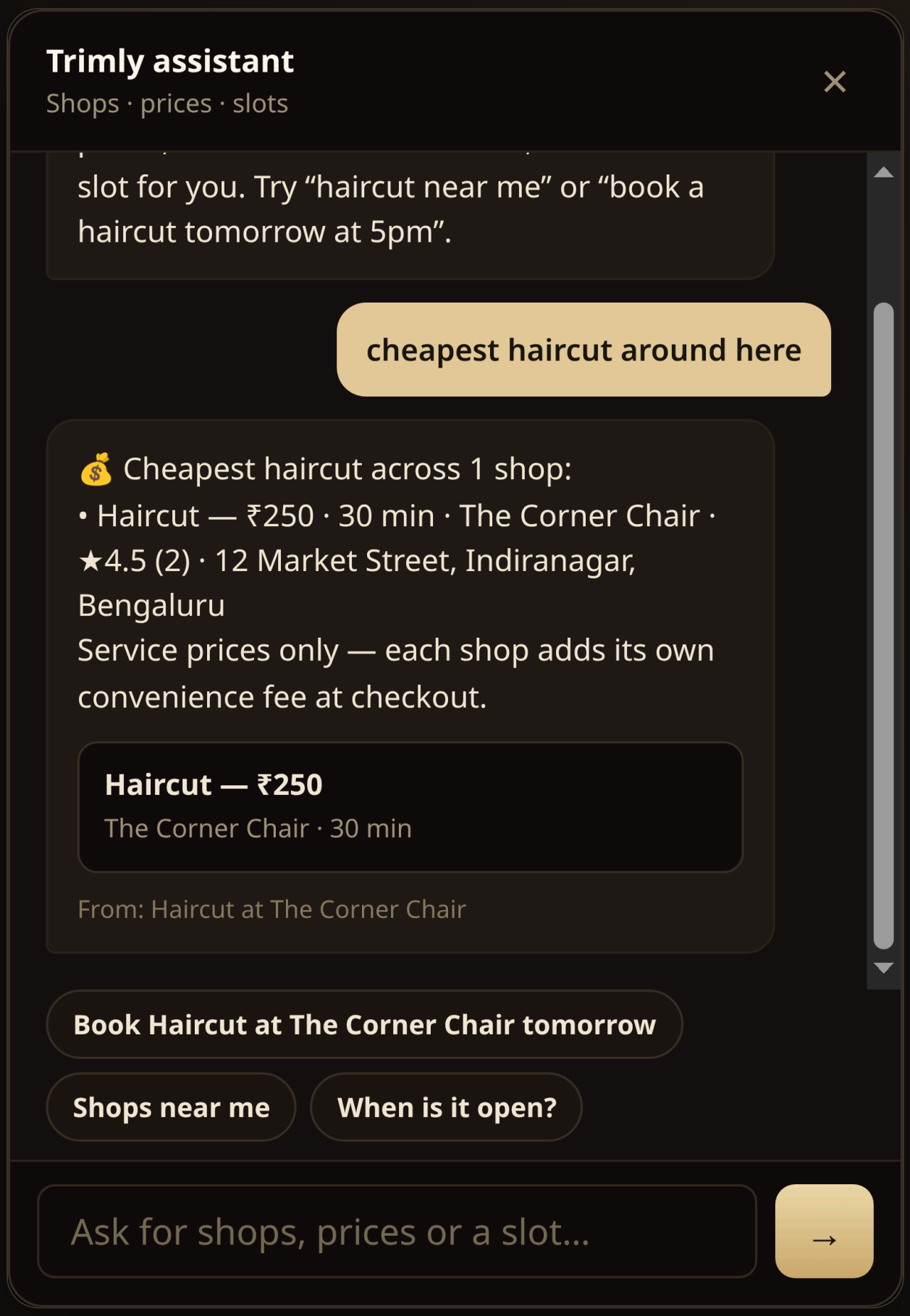
What gets retrieved

- → **1 document per shop** — menu, hours, rating, distance
- → **1 per service** — price and duration, straight from the table
- → **1 per hours sheet, per review** — text people actually wrote
- → **owner side:** bookings and payments from the last 30 days

> BM25 (k1 1.5, b 0.75) · rebuilt on every request · no API key · no index job · works offline

CUSTOMER SIDE

Prices, not prose

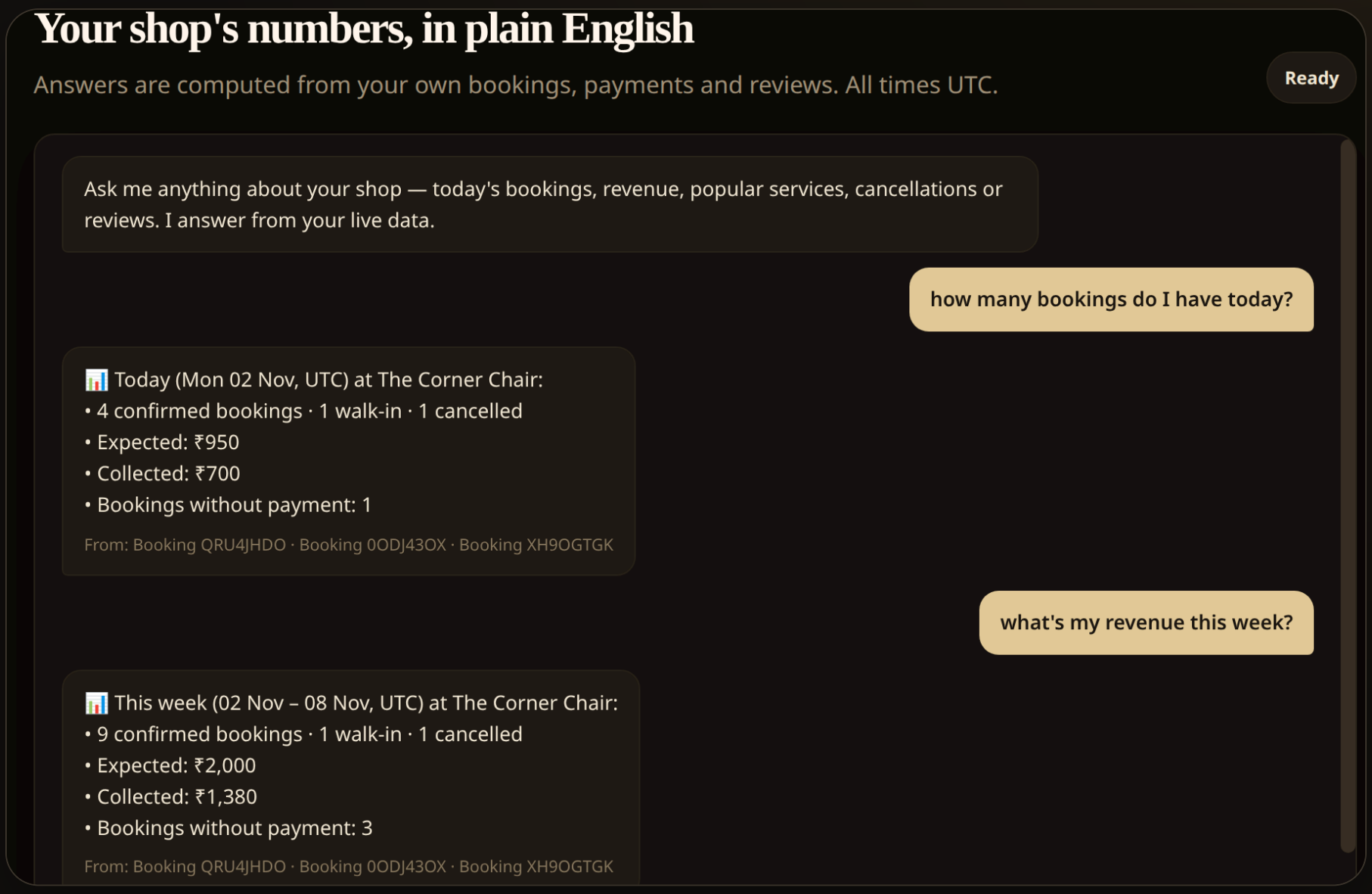


₹250 is `services.price_paise`. The line under the answer names the document it came from — and the card is tappable to book.

“cheapest haircut around here”

OWNER SIDE

Numbers, with receipts



Counts come from `bookings`, money from `payments`. The codes under each answer are the rows that produced it.

BOOKING

Booking stays boring

- → Slots come from the same availability engine as the web dialog — hours ∩ interval ∩ not booked ∩ future
- → Bookings go through the same `BookingService` the UI uses
- → The unique index on `(shop_id, starts_at)` is the final guard

Slot taken between read and write?

The 409 becomes “that slot just went — here are the next three openings.” No lost booking, no lie.

EVIDENCE

Why the numbers hold up

37
tests green, 13 of them for the assistants

0
API keys, model downloads or vector DBs

100%
of answers return the rows they were built from

1
bug caught by reading the output, not the pipeline

> Retrieval quality is a correctness feature here, not a tuning knob.

YOUR TURN

What would you ask your own shop data first?

I built this for a barbershop booking platform: two assistants, one retrieval layer, every answer traceable to a row.

The full write-up — architecture, request flow, and the "today vs this week" bug — is in the comments.

Trimly · FastAPI + SQLAlchemy + a hand-written BM25 index · no LLM in the answer path